

NEURAL HIVE

AI agents that work together, verified on Avalanche

Grant Proposal: Autonomous On-Chain Agents Track

Avalanche AI Innovation Grants

Requested funding: up to \$150,000 over 24 months

Website: <https://neural-hive.netlify.app>

Date: September 26, 2026

Submitted by: Saptarsi Halder and Manjith BC

Prepared for grant review purposes

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Problem Statement	3
3. Project Description & Use Cases.....	3
3.1 What We're Building.....	3
3.2 Why Avalanche, Why Now.....	4
3.3 Use Cases.....	4
4. Technical Architecture	5
4.1 System Overview	5
4.2 Core Coordination & Trust Algorithms.....	5
4.3 On-Chain Design	10
4.4 Proposed Technology Stack.....	11
5. Technical Roadmap & Milestones.....	11
6. Business Model & Sustainability	12
7. Budget & KPIs.....	13
7.1 Budget Breakdown	13
7.2 Key Performance Indicators.....	13
8. Team Background.....	14
9. Repos & Past Projects	14
10. Risks & Mitigations.....	15
11. Why Fund Neural Hive Now	15
Appendix A: Glossary	17
Appendix B: References.....	17

1. Executive Summary

Neural Hive is a coordination layer that lets many small AI agents work together instead of one company's model doing everything. A request comes in, Neural Hive breaks it into smaller steps, sends each step to the agent best suited for it, checks the answers it gets back, and pays everyone involved in HIVE, our token.

We are building this directly on Avalanche. Neural Hive does not launch its own chain. The agent registry, the reputation system, and the staking and payment logic all live as smart contracts on Avalanche's C-Chain, so every task we process adds real activity to Avalanche itself instead of pulling users onto a separate network.

This proposal targets the Autonomous On-Chain Agents track of the Avalanche AI Innovation Grants. We are asking for up to \$150,000 over 24 months to take Neural Hive from design to a working, audited product on Avalanche, with agents that can manage liquidity, follow predefined trading logic, and respond to market events on their own, while staying checkable instead of being a black box.

2. Problem Statement

AI is becoming as basic as electricity. But unlike electricity, most of the intelligence people and businesses rely on comes from a handful of big companies. To use it, you go through their API, pay their price, and trust that their interests match yours.

As models get better, the compute and money needed to build and run them keeps going up. That pushes real AI power further into the hands of whoever can afford the biggest data centers, so intelligence ends up more powerful, more concentrated, and more expensive at the same time. Compute, storage, and bandwidth all went the other way as they matured: cheaper and more spread out, not less.

On-chain, this gets even harder. A smart contract can't call a centralized AI API and just trust what comes back. So most "AI x crypto" projects today do one of two things: run the AI off-chain behind a company's backend that users have to trust, or keep AI away from anything involving real funds. Neither works long-term for an ecosystem like Avalanche's, especially with hundreds of new L1s launching, all needing the same things: risk scoring, moderation, summarization, anomaly detection, and on-chain agents that can actually act.

3. Project Description & Use Cases

3.1 What We're Building

Neural Hive has three parts. A coordination layer takes a request, breaks it into smaller steps, and sends each step to the agent or model best suited for it. A network of independent agents and compute providers does the actual work and gets paid for it. And a set of smart contracts on Avalanche, covering who's registered, who's trustworthy, and who owes whom what, ties the first two parts together without a company sitting in the middle taking a cut.

HIVE is the token that pays for all of this. Requesters spend it to get answers. Agents and compute providers earn it for good work. Anyone who wants to run an agent on the network also stakes HIVE, which backs the reputation and dispute system described in Section 4.

3.2 Why Avalanche, Why Now

We build directly on Avalanche's C-Chain instead of launching a separate chain. That matters for three reasons.

- Avalanche's Avalanche9000 upgrade cut C-Chain fees by around 96%. Neural Hive settles a lot of small payments, often just a few cents per task, many times a day. That only works if gas stays cheap, and Avalanche is one of the few chains where it already does.
- Staying on the C-Chain means every task Neural Hive runs is a transaction on Avalanche's main chain. We add activity to Avalanche's existing ecosystem instead of pulling users onto a chain of our own that nobody else has a reason to use.
- Avalanche's Interchain Messaging (ICM, the successor to Avalanche Warp Messaging and Teleporter) lets other Avalanche L1s call into our contracts on the C-Chain without needing a bridge. So a game chain or a DeFi chain running as its own Avalanche L1 can still use Neural Hive's agents, even though Neural Hive itself lives on the C-Chain.

Timing matters too. Avalanche now has hundreds of new L1s launching, and each one will eventually need risk scoring, moderation, or automated agents. Today, most would either build this in-house or plug in a centralized API. Neural Hive is a bet that it's better to build this once, on Avalanche, in the open, before that happens.

3.3 Use Cases

Autonomous on-chain agents (our main focus)

Neural Hive agents plug directly into Avalanche smart contracts and act on their own: rebalancing a liquidity position, following a trading strategy within limits the owner set, or responding to a price move or a governance event the moment it happens. Because every agent's output passes through the verification and reputation layers in Section 4, its actions can be checked after the fact instead of being trusted blindly.

Cross-chain requests through ICM

Any Avalanche L1 can send a request into Neural Hive's contracts on the C-Chain through ICM and get a verified answer back, paying in HIVE, with no bridge and no centralized backend to trust.

An open marketplace for models and compute

Model and GPU operators list what they can do and compete for tasks under the pricing rule in Section 4. Open competition, not one company's margin, is what keeps prices down.

Multi-step workflows for dApps

A request like “check this loan's risk, explain it in plain language, and flag it for review if needed” gets split into steps, routed to different agents, checked, and returned as one answer, instead of a dApp team wiring together three separate APIs and trusting each one on its own.

4. Technical Architecture

4.1 System Overview

A request comes in through our SDK or as an ICM message from another Avalanche L1. The coordination layer breaks it into steps (4.2.4) and matches each step to candidate agents by meaning, not just keywords (4.2.1). Most of the time, picking who actually does the work is left to fast selfish routing (4.2.2); when the network is under strain, it switches to a slower but smarter bandit-based assignment instead (4.2.3). For steps where a wrong answer is costly, we send the same step to several agents and combine their answers in a way one bad actor can't skew (4.2.5), weighted by reputation (4.2.6). Payment is settled with a pricing rule that rewards honesty (4.2.7). If agents still disagree after that, a voting process modeled on Avalanche's own consensus settles it (4.2.8). Section 4.2.9 covers when we go further and actually prove an answer was computed correctly.

None of the ideas below are new. We did not invent a secret algorithm. We picked well-tested tools from distributed systems, game theory, and machine learning, and put them together for a network of agents that don't know or trust each other ahead of time. A few of these, Krum, EigenTrust, VCG pricing, selfish routing, are well known in research circles but rarely combined in one working system. That combination is the actual contribution here.

4.2 Core Coordination & Trust Algorithms

Component	Algorithm	Reference	Why This One
Capability matching	HNSW search plus a learned scoring step	Malkov & Yashunin, 2018; Wang et al., 2024	Finds good candidates fast even with thousands of agents registered, and weighs price, load, and reputation, not just similarity

Component	Algorithm	Reference	Why This One
Fast load balancing	Selfish routing (Nash equilibrium), watched against a threshold	Roughgarden & Tardos, 2002	Lets tasks pick their own agent with no central decision needed, kept in check so a bad equilibrium can't quietly slow the whole network down
Smart fallback assignment	Contextual bandit (LinUCB)	Li, Chu, Langford & Schapire, 2010	Learns which agent is actually best for a task type from real outcomes; kicks in once selfish routing crosses its threshold
Breaking down requests	Step-by-step decomposition plus critical path scheduling	Standard AI planning and project scheduling	Finds the slowest chain of steps and prioritizes it so the whole request doesn't wait on the weakest link
Combining answers	Krum	Blanchard, El Mhamdi, Guerraoui & Stainer, 2017	Picks the most agreed-upon answer instead of an average, so one bad agent can't skew the result
Reputation	EigenTrust	Kamvar, Schlosser & Garcia-Molina, 2003	Trust spreads through the network the way PageRank ranks pages, cheap to run and hard to fake
Pricing	VCG mechanism	Vickrey, 1961; Clarke, 1971; Groves, 1973	Makes honest bidding the best strategy for agents, so prices reflect real cost
Settling disagreements	"Hive Snowball," repeated random sampling	Team Rocket, 2018 (Avalanche's own paper)	Reuses the same idea Avalanche's own consensus runs on, instead of importing an unrelated protocol
Checking work	Spot-checks by default, full proof for high-value calls	Optimistic rollup pattern; EZKL-style zkML	A full zero-knowledge proof on every call is still too slow and costly; this gives us a dial instead of an on-off switch

4.2.1 Semantic Capability Routing (HNSW plus a Learned Gate)

Every agent that joins Neural Hive publishes a small vector describing what it's good at, based on its declared skills and any benchmark scores it has. We index these with HNSW (Malkov and Yashunin, 2018), which finds the closest matches to an incoming task in roughly $\log(N)$ time instead of checking every agent one by one. That short list then gets re-ranked by a simple scoring step, similar in spirit to the Mixture-of-Agents idea (Wang et al., 2024), that also looks at current load, price, and reputation, not just how well the task and agent match on paper.

```
function ROUTE_CANDIDATES(task_embedding, k):
    candidates = HNSW_SEARCH(capability_index, task_embedding, k)
    for agent in candidates:
```

```

agent.score = GATE(task_embedding, agent.capability_vec,
                  agent.reputation, agent.price, agent.load)
return TOP_N(candidates, by = agent.score)

```

4.2.2 Selfish Routing with a Threshold

Running a central router for every single task adds delay, and at scale that delay adds up. So for most tasks, Neural Hive lets agents compete for work the selfish way: each agent advertises its current load and price, and each task simply goes to whichever qualified agent looks best to it at that moment, with no central decision in the loop. This is a routing game, and it settles into a Nash equilibrium: a state where no task would do better by switching to a different agent, given what everyone else is doing.

The catch, known since Roughgarden and Tardos's 2002 paper “How Bad Is Selfish Routing?,” is that a selfish equilibrium can end up worse than what a central planner would have chosen. This gap has a name: the price of anarchy. So we watch it. We track a simple proxy, the gap between the average wait time under selfish routing and an estimate of the best possible wait time. If that gap crosses a set threshold, meaning selfish routing is genuinely dragging performance down, we switch that class of traffic over to the slower but smarter bandit-based routing in 4.2.3, then switch back once things settle. In practice, the network is fast by default and only pays for central coordination when selfish behavior is actually hurting it.

```

function SELFISH_ROUTE(task, candidate_agents):
    for agent in candidate_agents:
        agent.perceived_cost = agent.advertised_load + agent.price
    return argmin(candidate_agents, by = agent.perceived_cost)

# measured continuously by the coordination layer:
poa_estimate = avg_actual_wait / estimated_optimal_wait
if poa_estimate > THRESHOLD:
    route_mode = "bandit" # fall back to LinUCB, section 4.2.3
else:
    route_mode = "selfish" # cheap, no central decision needed

```

4.2.3 Adaptive Task-Agent Assignment (Contextual Bandit, LinUCB)

When selfish routing crosses its threshold, or for higher-stakes tasks where we want more care from the start, Neural Hive switches to LinUCB (Li, Chu, Langford, and Schapire, 2010), a contextual bandit. Each agent gets a simple model that predicts how well it will do on a task based on the task's features. At each step, the router picks the agent with the best predicted result plus a small bonus for agents it hasn't tried much yet, then updates that model once it sees the real outcome. Over time this learns which agent is actually good at what, and adjusts on its own if a previously reliable agent starts slipping.

```

for each agent a in shortlist: initialize A_a = I, b_a = 0

```

```

given context  $x_t$  (task features):
  for each agent  $a$ :
     $\theta_a = \text{inverse}(A_a) \cdot b_a$ 
     $p_a = \theta_a^T x_t + \alpha \cdot \sqrt{x_t^T \cdot \text{inverse}(A_a) \cdot x_t}$ 
    choose  $a^* = \text{argmax}(p_a)$ 
    observe reward  $r_t$  (quality, latency, cost)
     $A_{a^*} += x_t x_t^T$ 
     $b_{a^*} += r_t \cdot x_t$ 

```

4.2.4 Breaking Down Requests (Step Graph plus Critical Path)

A request like “check this contract, summarize the risk, and suggest a fix” is not one task, it's three, and the second and third depend on the first finishing. We break requests like this into a graph of smaller steps, sort them into a valid order, and work out which chain of steps is the slowest one, the critical path. We then give our fastest agents to that chain, since that is what actually decides how long the whole request takes.

```

G = DECOMPOSE(request)      # expand request into a graph of steps
order = TOPOLOGICAL_SORT(G)
for node in order:
  node.earliest_start = max(pred.earliest_finish for pred in preds(node), default = 0)
  node.earliest_finish = node.earliest_start + estimated_duration(node)
critical_path = BACKTRACK_LONGEST_PATH(G)
ALLOCATE_FASTEST_AGENTS(critical_path)
EXECUTE(G, order)

```

4.2.5 Combining Answers Safely (Krum)

Because anyone can join as a compute provider, some agents will occasionally be wrong, and a few might be actively dishonest. For tasks where that matters, we send the same step to several agents and combine their answers with Krum (Blanchard, El Mhamdi, Guerraoui, and Stainer, 2017) instead of just averaging or taking a majority vote. Krum picks the answer that sits closest to the most other answers and ignores the ones that are far from everyone else. That means one bad actor with an extreme answer can't drag a simple average off course.

```

function KRUM(outputs, f):
  n = length(outputs)
  for i in 0..n-1:
    d = sort([distance(outputs[i], outputs[j]) for j != i])
    scores[i] = sum(d[0 : n-f-2])
  return outputs[argmin(scores)]

# distance() compares answers in embedding space for text or
# structured output, or plain numeric distance for numbers

```

4.2.6 Reputation That's Hard to Fake (EigenTrust)

A simple star rating is easy to game with a few colluding accounts. EigenTrust (Kamvar, Schlosser, and Garcia-Molina, 2003) fixes this by spreading trust through the whole network, the same way Google's PageRank ranks a page by who links to it, not just by how many clicks it gets. An agent's real reputation ends up reflecting whether other trustworthy participants trust it, which is a lot harder and more expensive to fake than a handful of good reviews.

```
C = row_normalize(local_trust_matrix)
t = uniform_vector(n)
repeat until convergence:
    t = transpose(C) . t
    t = t / ||t||_1
return t # global reputation score, feeds routing weight and stake terms
```

4.2.7 Fair Pricing (VCG Mechanism)

When several agents can do the same job, we need a way to pick one and set a fair price without giving anyone a reason to lie about their real cost. A VCG mechanism (Vickrey, 1961; Clarke, 1971; Groves, 1973) picks the agent that creates the most value overall and pays the winner based on what its presence cost everyone else, not its own bid. That makes bidding your real cost the best strategy, which avoids the games agents would otherwise play in a market where everyone remembers everyone else.

```
function VCG_ALLOCATE(bids):          # simplified single-item case
    winner = argmin(bid.cost for bid in bids)
    runner_up = min(bid.cost for bid in bids if bid != winner)
    payment = runner_up # winner is paid the 2nd-lowest bid, not its own
    return winner, payment

# larger batches use the general VCG rule with a Clarke-pivot
# payment for each winner
```

4.2.8 Settling Disagreements (“Hive Snowball”)

If independent checkers still disagree on the right answer after Krum, we need a fast way to settle it without a slow, full network vote. We borrow the idea behind Avalanche's own consensus, from Team Rocket's 2018 paper “Snowflake to Avalanche”: ask a small random group for their opinion, repeat with new random groups, and let confidence build each time the answer stays the same. Once one answer wins enough rounds in a row, it's final. This reuses a method Avalanche already trusts at the chain level, just applied one layer up, instead of bringing in some unrelated consensus protocol.

```
preference = KRUM(outputs, f)
confidence = 0
repeat until confidence >= beta or round_limit_reached:
    sample = random_subset(verifier_committee, k)
```

```

votes = [v.evaluate(preference) for v in sample]
majority = MAJORITY(votes)
if majority == preference: confidence += 1
else: preference = majority; confidence = 1
finalize(preference)

```

4.2.9 Checking Work Honestly, Toward zkML

Proving every single answer with a zero-knowledge proof is still too slow and expensive to do for every task today. So by default, we trust the Krum-checked, reputation-weighted answer. For higher-value tasks, or ones from an agent that doesn't have much reputation yet, we sometimes ask independent agents to redo the work, and for the most important on-chain calls, we get a full zkML proof instead, likely using EZKL-style tooling. If an answer turns out to be wrong, the agent that gave it loses its stake. This gives us a dial between cost and certainty instead of an all-or-nothing choice on day one. We're planning full zkML as something to pilot later in the roadmap, once the rest of the system has proven itself.

4.3 On-Chain Design

Everything below is a normal smart contract on Avalanche's C-Chain. None of it requires launching a separate chain.

- **Capability Registry:** tracks each agent's declared skills, price, and current stake. Our HNSW index (4.2.1) is built from this and checked against it regularly.
- **Reputation Registry:** stores each agent's EigenTrust score (4.2.6), updated on a regular schedule from trust signals submitted by requesters and other agents.
- **Staking and settlement contract:** holds each agent's stake, runs the VCG payment rule (4.2.7) once a task is done, and slashes stake when an agent is proven wrong.
- **ICM connection:** lets other Avalanche L1s send requests into these contracts and get answers back, without a bridge.

Illustrative HIVE tokenomics (a starting point, not final)

Allocation	Share	Purpose
Agent and compute rewards	40%	Ongoing payments to agents and compute providers for verified work
Ecosystem and community	15%	Onboarding, integrations, developer support
Founders and team	18%	4-year vesting, 1-year cliff
Treasury reserve	15%	Runway, audits, contingency
Public distribution and liquidity	12%	Initial liquidity and public access to HIVE

This split is a starting point, not a final design. HIVE is planned as a standard token contract on Avalanche's C-Chain, not a native gas token for a separate chain.

4.4 Proposed Technology Stack

Layer	Proposed Technology	Notes
Smart contracts	Solidity, deployed directly on Avalanche's C-Chain	Registry, reputation, staking and settlement contracts
Coordination layer	Rust or Go services, with Python for the ML pieces	HNSW index, selfish and bandit routing, Krum, EigenTrust
Agent framework	Open, graph-based multi-step orchestration	A standard interface so outside agents can register regardless of what they're built with
Interchain	Avalanche Interchain Messaging (ICM)	Lets other Avalanche L1s call into our C-Chain contracts
Verification	Optimistic checks now, EZKL-style zkML piloted later	Matches the cost and certainty dial in 4.2.9

5. Technical Roadmap & Milestones

Timelines below are counted in months after the grant is released, not fixed calendar dates, since we don't know exactly when funding would land. Total project length is 24 months.

Phase 1, Months 1 to 6: Foundation and testnet basics

- Deliverables: finished spec; Capability Registry, staking, and settlement contracts live on Avalanche's Fuji testnet; HNSW routing (4.2.1) and selfish routing with a threshold (4.2.2) working for 3 to 5 task types.
- Resources: 2 protocol engineers, 1 smart contract engineer, part-time audit consulting.
- Funding released: \$52,500 (35%).

Phase 2, Months 7 to 12: Trust and verification

- Deliverables: EigenTrust reputation (4.2.6) live; Krum aggregation (4.2.5) for higher-value tasks; LinUCB fallback routing (4.2.3) live; VCG pricing (4.2.7); at least 15 outside agent or compute operators onboarded to testnet.
- Resources: 2 protocol engineers, 1 ML engineer, developer relations support for onboarding.
- Funding released: \$37,500 (25%).

Phase 3, Months 13 to 18: Mainnet prep and business model

- Deliverables: Hive Snowball dispute resolution (4.2.8) live; protocol fee mechanism (Section 6) built and tested; full external security audit; mainnet contracts deployed on Avalanche's C-Chain; ICM integration live so other Avalanche L1s can call in.
- Resources: 2 engineers, external audit firm, legal review of the token and fee model.
- Funding released: \$30,000 (20%).

Phase 4, Months 19 to 24: Public launch and testing

- Deliverables: public launch on Avalanche C-Chain mainnet; load testing under real traffic; zkML pilot (4.2.9) for a handful of high-value calls; developer SDK and docs; at least 5 real integrations with other projects or dApps.
- Resources: 2 engineers, 1 developer relations or community person, continued audits if needed.
- Funding released: \$30,000 (20%).

Funding is released in four tranches tied to the milestones above, adding up to \$150,000 total. No tranche is paid before its milestone is met.

6. Business Model & Sustainability

A grant gets Neural Hive built. It doesn't keep it running. Here is how we plan to cover costs once the grant period ends.

- Protocol fee: every completed task pays a small fee on top of what the agent earns, somewhere around 5 to 10%, tuned during testnet. That fee goes to the treasury and covers ongoing development, audits, and infrastructure.
- Premium verification: tasks that request the full zkML-checked tier (4.2.9) pay more than a task that accepts the default optimistic check. We charge more for the certainty that costs us more to provide.
- Enterprise support: teams that want a dedicated pool of agents, custom guarantees, or a private deployment pay for that directly, similar to how oracle networks like Chainlink charge for enterprise integrations.
- Staked capital: agent stake sits in the settlement contract and, when it isn't backing an active dispute, can be put into a low-risk yield strategy on Avalanche, adding a modest amount of treasury income. We'll stay conservative here. This is a plan, not a promise.

Our target is fee-based break-even, meaning treasury income covers baseline running costs, not new feature work, within 12 months of mainnet launch. That falls inside the 24-month window of this proposal, in Phase 4.

7. Budget & KPIs

7.1 Budget Breakdown

Category	24-Month Estimate	Notes
Core engineering (contracts and coordination layer)	\$65,000	Phases 1 to 4; the largest line item
ML and mechanism design	\$22,000	Bandit tuning, reputation, auction, and selfish-routing threshold work
Security audit	\$25,000	One full external audit before mainnet
Infrastructure (testnet and mainnet)	\$12,000	Nodes, indexers, monitoring, over 24 months
Developer relations, community, and docs	\$10,000	Onboarding outside agents and integrators
Legal and compliance	\$8,000	Token and fee model review before mainnet
Contingency	\$8,000	Buffer for the unexpected
Total	\$150,000	Released in four tranches tied to the roadmap phases

7.2 Key Performance Indicators

These are targets for the funding period, not results we've already hit:

KPI	Target	By When
Registered agents and compute providers	15, then 40	End of Phase 2; end of Phase 4
Testnet completed tasks	200 or more per day	End of Phase 2
Routing latency, cache-hit path (p95)	Under 300 ms	End of Phase 3
Full multi-step pipeline latency (p95)	Under 2 seconds	End of Phase 3
Cost per task versus a centralized API baseline	60% or less (target, to be checked against real numbers)	End of Phase 4
Fault tolerance per verification committee	Correct result even with up to about a third of agents faulty	Ongoing from Phase 2

KPI	Target	By When
Outside integrations	5 or more	3 months after launch
Partner Avalanche L1s reached through ICM	2 or more	End of Phase 4
Fee-based break-even	Treasury income covers baseline running costs	Within 12 months of mainnet launch

8. Team Background

Neural Hive is founded by two people.

Saptarsi Halder, Founder and Chief On-Chain Officer, holds an M.Sc. in Computer Science Engineering from Sapienza University of Rome. He previously worked as a QA engineer and was a founding member of Magic Maps, with hands-on experience across software engineering, distributed systems, blockchain, and AI.

Manjith BC, Co-Founder and Chief Consultant, is an Assistant Professor at IIIT Kottayam, with a background in FPGA design, Advanced Cryptography, quantum computing, and AI.

We're a small team by design. Part of this grant goes toward hiring the engineers listed in the roadmap above, against specific milestones, not hiring ahead of need.

9. Repos & Past Projects

Neural Hive Website (Previous Repository)

<https://github.com/neural-hive/neural-hive-web>

HIVE Smart Contract Set

<https://github.com/neural-hive/hive-smart-contract-set>

Block Borrower Smart Contracts

Previous project demonstrating smart contract development and blockchain capabilities.

<https://github.com/sap200/block-borrower-contracts>

Block Borrower Frontend

<https://github.com/sap200/block-borrower>

Block Borrower Relayer

<https://github.com/sap200/blockrelayer>

Quantum Key Distillation

Quantum computing and key distillation research project.

<https://github.com/manjithbc/QKD-Key-distillation-cow-dsp-DPR>

10. Risks & Mitigations

Risk	Mitigation
Fake reputation through many colluding accounts	EigenTrust's transitive trust plus staking and slashing make manufactured reputation far more expensive than gaming a simple rating.
Full zkML checking is still too slow and costly for real-time use	Optimistic checks plus spot-checking cover most traffic today; zkML is a Phase 4 pilot on high-value calls only, not a day-one requirement.
Regulatory risk around the token	Conservative utility design, no promised profit share, and legal review of the token and fee model before mainnet.
Cold start, no agents and no demand at launch	Phase 1 seeds the registry with a small set of open models we run ourselves; developer relations support through this program helps bring in real outside agents.
Gas costs on the C-Chain rise unexpectedly	We batch settlements where we can and keep an eye on gas trends, since our margins on small tasks depend on fees staying low.
Small team	Two founders today. Grant funds go directly toward hiring against the roadmap above, not speculative expansion.

11. Why Fund Neural Hive Now

Two things are happening at once. AI is consolidating around a few large companies right as it becomes something everyone depends on. And Avalanche has just made it cheap enough to settle lots of small on-chain payments that a network like this is actually viable to run on the C-Chain itself, without needing a chain of our own.

We think the right response to the first problem is to use the second. Build the agent layer where it can interact with Avalanche smart contracts today, verified and checkable, instead of waiting for someone else to do it behind closed doors.

We're asking for up to \$150,000 over 24 months to build this in the open, on Avalanche, with two founders who are putting their own track record behind it.

Appendix A: Glossary

- Avalanche9000 (Etna upgrade): Avalanche's late-2024 network upgrade. It cut fees on the C-Chain by around 96% and also made launching separate Avalanche L1s much cheaper, though Neural Hive is not launching one.
- ICM (Interchain Messaging): Avalanche's native cross-chain messaging system, the successor to Avalanche Warp Messaging and Teleporter.
- Nash equilibrium: a state in a system with many independent decision-makers where nobody can do better by changing their choice alone, given what everyone else is doing.
- Price of anarchy: how much worse a system performs when everyone acts in their own interest compared to what a central planner could achieve.
- Byzantine fault tolerance: a system's ability to keep working correctly even when some participants behave arbitrarily or dishonestly.
- Sybil resistance: resistance to one attacker creating many fake identities to gain outsized influence over a network.

Appendix B: References

- Team Rocket, "Snowflake to Avalanche: A Novel Metastable Consensus Protocol Family," 2018.
- Roughgarden, T., and Tardos, É., "How Bad Is Selfish Routing?," Journal of the ACM, 2002.
- Kamvar, S., Schlosser, M., and Garcia-Molina, H., "The EigenTrust Algorithm for Reputation Management in P2P Networks," WWW 2003.
- Blanchard, P., El Mhamdi, E. M., Guerraoui, R., and Stainer, J., "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," NeurIPS 2017.
- Li, L., Chu, W., Langford, J., and Schapire, R., "A Contextual-Bandit Approach to Personalized News Article Recommendation," WWW 2010.
- Malkov, Y., and Yashunin, D., "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," 2018.
- Wang, J., et al., "Mixture-of-Agents Enhances Large Language Model Capabilities," 2024.
- Vickrey, W. (1961); Clarke, E. (1971); Groves, T. (1973): the foundational VCG mechanism design papers.
- Avalanche documentation on the Avalanche9000 upgrade and Interchain Messaging: docs.avax.network.